Verifiable Provenance for Agentic AI: Securing the Software Supply Chain in the Era of Autonomous Development

Introduction: The Emerging Crisis of Opaque Provenance

The Paradigm Shift: From AI Co-pilot to Autonomous Agent

The landscape of software development is undergoing a tectonic shift, driven by the rapid evolution of artificial intelligence. For years, AI has served as an assistive technology, a co-pilot augmenting the capabilities of human developers. Tools like the initial iterations of GitHub Copilot integrated directly into the developer's workflow, offering contextual code completions and function suggestions, but always leaving the final agency—the decision to accept, reject, or modify—in the hands of the human operator.[1] In this model, the chain of authorship and responsibility, while assisted by machine intelligence, remained clearly anchored to the developer.

A new paradigm, however, is rapidly supplanting this model: the era of the autonomous agent. This transformation is epitomized by the emergence of "agentic workflows," a concept actively being explored by research divisions like GitHub Next and implemented in commercially available tools such as Google's Gemini CLI GitHub Actions.[2] These are not mere assistants; they are designed to function as autonomous agents and on-demand collaborators. Triggered by repository events or direct mentions, these AI systems can independently execute complex, multi-step tasks that were once the exclusive domain of human engineers. These tasks include intelligent issue triage, where an agent analyzes, labels, and prioritizes incoming bug reports; accelerated pull request reviews, where it provides instant feedback on code quality and style; and even direct code implementation, where it can

be instructed to "write tests for this bug" or "implement the changes suggested above".[1]

This leap from suggestion to autonomous action represents a fundamental inversion of the traditional developer-tool relationship. Historically, a tool, whether a compiler or a linter, was a deterministic extension of a developer's explicit command. Its actions were directly and unambiguously traceable to a human's intent. Even assistive AI maintained this link, as the human remained the final arbiter and author. Agentic workflows sever this connection. A developer now provides a high-level, natural language goal—"improve documentation," "refactor this module for efficiency"—and the agent is responsible for formulating the sub-tasks, executing the steps, and producing the final artifact. The resulting code commit or issue comment is no longer the direct product of a human's intellectual process but the output of an opaque, non-deterministic system. This inversion is the foundational cause of a looming crisis in software integrity, as it erases the observable and verifiable link between human intent and low-level implementation.

Defining the "Provenance Gap"

At the heart of this crisis is the emergence of a critical "provenance gap." Provenance, a concept with deep roots in scientific computing and data management, is formally defined as a record describing the people, institutions, entities, and activities involved in producing, influencing, or delivering a piece of data or a thing.[6] In the context of software engineering, it is the intellectual and operational lineage of code—a documented history that answers the fundamental questions of who wrote it, what data and requirements it was based on, and what processes were used to create and validate it.[8] This lineage is the bedrock of trust, reproducibility, and accountability in software development.

The provenance gap is the systematic loss of this lineage when an autonomous AI agent becomes a primary author or contributor. The "black box" nature of many advanced AI models means that their decision-making processes are inherently opaque, even to their own creators.[10] When an agent produces a novel algorithm or a critical bug fix, a series of vital questions become unanswerable. What specific data sources—from public repositories, internal documentation, or the project's codebase—informed the agent's final decision? What intermediate reasoning steps, failed attempts, or "thoughts" did the agent cycle through before arriving at its solution? Which specific model parameters, tool outputs, or context files provided in a

GEMINI.md file were the decisive factors that influenced the final code commit or PR comment? [User Query]. This void of information is the provenance gap, and it renders the outputs of agentic workflows scientifically unverifiable and fundamentally untrustworthy.

Thesis: Beyond Observability to Verifiability

Current attempts to address this opacity primarily focus on *observability*. For instance, the Gemini CLI's integration with OpenTelemetry provides a mechanism for streaming logs and metrics, offering a real-time view into the actions an agent is performing.[4] This is a necessary and valuable first step, akin to having a flight data recorder that logs the aircraft's altitude, speed, and control surface positions. It can tell an investigator

what happened in the moments leading up to a failure. However, it cannot prove that the data in the recorder has not been tampered with, nor can it fully explain the complex interplay of factors that led to the pilot's (or agent's) decision.

Observability is insufficient for the high-stakes environment of software development, where the integrity of the final product has profound security, legal, and financial implications. The challenges of regulatory compliance, intellectual property protection, and research integrity demand a much higher standard: *cryptographic verifiability*. This requires the creation of a tamper-evident, non-repudiable record that not only documents what happened but also provides cryptographic proof of who (which specific agent instance) performed the action, why (based on what verifiable inputs and reasoning steps), and that the provenance record itself is authentic and has not been altered.

This report posits that bridging the provenance gap requires a multi-layered architectural framework that moves beyond simple logging. It necessitates the integration of standardized data provenance models like W3C PROV, the adaptation of software supply chain security principles such as SLSA (Supply-chain Levels for Software Artifacts), and the deployment of cryptographic primitives from the world of decentralized identity. Only through this holistic approach can a truly verifiable and trustworthy AI supply chain be established, transforming provenance from a passive, after-the-fact record into an active, verifiable asset that can be used to enforce policy, ensure compliance, and build lasting trust in the era of the autonomous developer.

The Rise of the Autonomous Developer: State of Agentic Workflows on GitHub

The theoretical promise of autonomous AI agents in software development is rapidly being realized through concrete implementations and research initiatives on platforms like GitHub.

Two prominent examples, GitHub's own "Agentic Workflows" research demonstrator and Google's commercially available "Gemini CLI GitHub Actions," offer a clear view of the current state-of-the-art and the architectural choices that are shaping this new landscape.

GitHub's Vision: "Agentic Workflows" as Natural Language Programming

GitHub Next, the company's research and development arm, is exploring a future where repository-level operations are programmed not with scripts but with plain English. Their "Agentic Workflows" project is a research demonstrator framed as a form of "natural language programming".[2] Instead of writing complex scripts using the GitHub API, a developer describes the desired behavior—such as "Continuously scan code and docs for accessibility issues and open PRs with suggested fixes"—in a simple markdown file stored within the

.github/workflows/ directory.[2]

A central pillar of this vision is the "Actions-first" design philosophy. The natural language markdown file is not interpreted at runtime. Instead, it is *compiled* into a standard, executable GitHub Actions YAML file, which is often saved as a .lock.yml file.[2] This compilation is performed by an agentic "engine," which can be powered by various Large Language Models (LLMs) such as Anthropic's Claude Code or OpenAI's Codex.[13] This approach is presented as a significant advantage for transparency and auditability. Because the final artifact that gets executed by the GitHub Actions runner is a familiar, source-controlled YAML file, organizations can leverage their existing tools and practices for auditing, versioning, and security scanning.[2] The design explicitly aims for model and engine independence, allowing teams to swap the underlying LLM without rewriting their natural language workflows, which underscores the need for provenance standards that are not beholden to a single AI vendor.[2]

While this design choice provides transparency at the point of execution, it simultaneously creates a significant blind spot at the point of creation. The compilation from natural language to YAML is a non-deterministic process. The resulting YAML is not just a function of the markdown input; it is also influenced by the specific LLM engine used, its version, its internal state, and any system-level prompts or guardrails applied by the platform. The generated .lock.yml file shows *what* will be executed but completely erases the *why*—the complex reasoning process the LLM undertook to translate the high-level goal into concrete steps. Auditing the YAML file is akin to auditing a compiled binary without access to the source code or the specific compiler version and flags used to create it. This "compilation" step, therefore, becomes a provenance black hole, swallowing the very intellectual lineage that is essential for true understanding and trust.

Google's Implementation: Gemini CLI GitHub Actions

Moving from research to production, Google has launched Gemini CLI GitHub Actions, a powerful suite of tools marketed as an "AI coding teammate" for repositories.[3] This integration operates both as an autonomous agent, triggered by events like new issues and pull requests, and as an on-demand collaborator that can be invoked directly within a conversation by mentioning

@gemini-cli.[3]

The system is designed to leverage the full context of a project, including its entire codebase, to perform its tasks with a high degree of relevance and accuracy.[3] It offers three primary workflows out-of-the-box: intelligent issue triage, which analyzes and labels new issues; accelerated pull request reviews, providing instant feedback on code changes; and open-ended collaboration for tasks like writing tests or implementing features.[4] These workflows are open-source and customizable, allowing teams to tailor the agent's behavior to their specific needs.[3]

A key focus of the Gemini CLI offering is enterprise-grade security. A significant feature is its use of Google Cloud's Workload Identity Federation (WIF) for authentication.[3] WIF is a modern, secure mechanism that allows GitHub Actions to authenticate with Google Cloud services without the need for long-lived, static API keys. Instead, it uses a token exchange process to grant short-lived, scoped credentials for each workflow run, drastically reducing the risk of credential compromise.[4] For observability, the system integrates with OpenTelemetry, enabling teams to stream logs and metrics to their preferred platforms to monitor usage and debug complex workflows.[4]

However, the robust security model presented by WIF highlights a critical distinction between two separate concepts: authentication and attestation. WIF is an excellent solution for *authentication*—it cryptographically proves that an agent *is who it says it is* and has the right to perform an action. It ensures that the entity making an API call is a legitimate GitHub Action from a trusted repository. But it does nothing to address *attestation*—the ability to prove that an agent *did what it says it did, based on a specific set of inputs and an auditable reasoning process*. Once the agent is securely authenticated via WIF, it operates as a trusted entity. If that trusted agent, due to a model hallucination or biased training data, generates insecure code or makes a flawed decision, WIF provides no mechanism to trace that error back to its root cause. The audit trail from WIF can prove that gemini-cli-action-run-123 was responsible for a specific commit, but it cannot prove *why* that action generated that specific line of code. The security model, therefore, is incomplete. It secures the channel of communication but

fails to secure the content and logic of the communication itself. This necessitates a separate, complementary framework for the cryptographic attestation of the agent's internal processes.

The Black Box Dilemma: Quantifying the Risks of Unverifiable AI Contributions

The "provenance gap" created by autonomous AI agents is not a theoretical concern; it introduces a spectrum of tangible risks that undermine the integrity, security, and legality of the software development lifecycle. Each contribution from an opaque AI system is a potential liability, transforming what might have been simple technical debt into a complex web of legal, financial, and reputational exposure. This systematic conversion of operational uncertainty into unbounded risk necessitates a deeper examination of the specific domains it affects.

Scientific and Research Integrity

In the realm of scientific and research computing, provenance is not a luxury but a foundational requirement. For decades, scientific workflow systems have been designed with the explicit goal of capturing detailed provenance to ensure the traceability, reproducibility, and quality assessment of experimental results.[7] This meticulous record-keeping allows researchers to verify findings, build upon previous work, and trust the data products generated by complex computational pipelines.[17]

Agentic AI threatens to dismantle this cornerstone of the scientific method. When an AI agent is used to analyze data, propose a hypothesis, or write the code for a simulation, its output becomes part of the scientific record. If that agent produces a novel insight or a critical bug fix, the inability to trace its inputs, model parameters, and internal reasoning makes the result scientifically unreproducible. An external researcher cannot verify the finding because the exact conditions that led to it are unknown and potentially unrecoverable. This is an unacceptable state of affairs in any high-assurance domain, from academic research to the development of software for medical devices or aerospace systems, where the reliability of every component must be rigorously verifiable.

Furthermore, the well-documented phenomenon of AI "hallucination"—where a model generates content that is plausible and syntactically correct but factually wrong or logically flawed—poses a severe threat.[10] An agent might generate code that compiles without errors and passes basic tests but contains a subtle, critical flaw that only manifests under specific

production conditions. Without a provenance trail to audit the agent's reasoning, debugging such an issue becomes a near-impossible task, eroding the reliability of the entire system.

Compliance and Legal Exposure

The introduction of opaque AI agents into software development workflows creates significant legal and regulatory liabilities, particularly under emerging data protection and AI-specific legislation.

- **GDPR and the "Right to Explanation"**: The European Union's General Data Protection Regulation (GDPR) establishes strong rights for individuals regarding their personal data. Articles 13, 14, and 15 grant data subjects the right to receive "meaningful information about the logic involved" in automated decision-making processes that produce legal or similarly significant effects.[18] An agentic workflow that automatically triages a user's bug report (which may contain personal data) or rejects a pull request could easily fall under this definition. If a user challenges such a decision, the organization deploying the agent is legally obligated to provide an explanation. An opaque system makes this impossible, creating a direct compliance failure and exposing the organization to fines of up to 4% of its annual global turnover.[20]
- **EU AI Act and Auditing Requirements**: The forthcoming EU AI Act will impose even more stringent requirements, particularly for systems classified as "high-risk".[20] The Act mandates exhaustive documentation, traceability, and the ability to conduct audits of AI systems' data, logic, and performance. Established auditing frameworks, such as COBIT and the GAO AI Accountability Framework, already emphasize the need for clear governance, data lineage, and performance monitoring as prerequisites for accountability.[23] An agentic workflow without a verifiable provenance trail would fail to meet these standards, potentially blocking a company's ability to operate in the European market.
- **Accountability and Liability Vacuum**: In the event of a failure—an AI-generated security vulnerability that leads to a data breach, or a flawed code change that causes a major service outage—opaque provenance creates a legal vacuum. Without a clear, verifiable chain of causality, assigning liability becomes a contentious and complex process. Is the responsible party the end-user who wrote the high-level prompt, the developer of the agentic tool, the provider of the underlying LLM, or the owner of the data on which the model was trained? This ambiguity not only complicates legal proceedings but also hinders the ability to learn from failures and prevent their recurrence.

Intellectual Property and Authorship

The legal frameworks governing intellectual property (IP) were built around the concept of human creation, and the rise of AI-generated content challenges these foundations, creating significant uncertainty for businesses.

- **Copyright and Authorship**: In most major jurisdictions, including the United States, copyright protection is granted only to works with a human author.[24] The U.S. Copyright Office has explicitly stated that when an AI technology determines the expressive elements of its output, the resulting material is not the product of human authorship and is therefore not protected by copyright.[25] This means that code generated autonomously by an agent may fall into the public domain. For a company building a proprietary software product, the inclusion of uncopyrightable, AI-generated code creates a critical vulnerability in its IP portfolio, potentially rendering key components of its product unenforceable against competitors.[26]
- **License Contamination and Infringement**: Generative AI models are trained on vast datasets scraped from the internet, which inevitably include large volumes of open-source code governed by a variety of licenses (e.g., GPL, AGPL, MIT).[24] There is a significant risk that an agent, having learned from this data, will generate code that is a derivative work of a copyleft-licensed project. If this code is incorporated into a proprietary product, it could trigger the viral clauses of the copyleft license, potentially obligating the company to release its entire product as open source. This risk of "license laundering" or inadvertent infringement is impossible to mitigate without a detailed provenance record that can trace the origins of the AI-generated code and demonstrate that it is not a derivative of restricted training data.
- **Trade Secret Exposure**: The use of agentic tools also presents a risk to a company's own trade secrets. If developers, in their prompts or by providing context files, include proprietary algorithms, business logic, or other confidential information, that data could be ingested by the AI model.[26] Depending on the terms of service of the AI provider and the architecture of the system, this could lead to the information being used to train future models or being inadvertently exposed to other users, resulting in a catastrophic loss of trade secret protection.

Security, Debugging, and Trust

Beyond the legal and compliance risks, the lack of provenance has immediate, practical consequences for the day-to-day operations of a software development team.

- **Debugging and Root Cause Analysis**: Multi-agent systems are inherently complex.

Their behavior can be emergent, and failures often cascade in unpredictable ways, making root cause analysis a formidable challenge.[28] When the internal reasoning path of each agent is an opaque black box, this challenge becomes an impossibility. A developer faced with a bug in AI-generated code has no "call stack" to inspect, no logical flow to trace. They are left with the frustrating and inefficient task of trying to reverse-engineer the agent's "intent" from its output alone.

- **The AI Supply Chain Attack Surface**: The entire lifecycle of an AI model, from data sourcing and preparation to training, deployment, and monitoring, constitutes a new and complex "AI supply chain".[30] This supply chain is vulnerable to a new class of attacks, such as data poisoning (maliciously altering training data to create backdoors in the final model) and model-centric attacks (directly tampering with model weights).[30] Without a verifiable provenance trail for both the data and the models used by an agent, detecting the origin point of a sophisticated supply chain attack is infeasible.
- **Erosion of Developer Trust**: Ultimately, a system that is not transparent, auditable, or verifiable cannot be trusted.[31] If developers cannot be confident in the quality, security, and legal integrity of the code generated by their "AI teammates," they will be hesitant to adopt them for critical tasks. The productivity gains promised by agentic AI will remain unrealized if every AI contribution must be painstakingly reviewed and validated by a human, defeating the purpose of the automation. Trust is the currency of collaboration, and opaque systems are fundamentally untrustworthy.

Foundational Pillars for a Verifiable AI Supply Chain

To construct a robust solution to the provenance crisis, it is necessary to move beyond monolithic approaches and instead build a framework upon several complementary technological pillars. Each pillar addresses a distinct aspect of the problem—the data model, the process integrity, and the cryptographic trust foundation. By integrating these components, a comprehensive and verifiable AI supply chain can be realized.

Pillar 1: Data-Level Provenance with W3C PROV and PROV-AGENT

The first pillar is a standardized, expressive language for describing provenance. The World Wide Web Consortium (W3C) has already established this with its Provenance Data Model (PROV-DM). PROV-DM is the international standard for provenance information, providing a domain-agnostic, conceptual data model for representing the lineage of any artifact.[6] It is

built upon three core concepts:

- **Entity**: A physical, digital, or conceptual thing. In our context, an entity could be a source code file, a dataset, a pull request, or a specific block of AI-generated code.
- **Activity**: A process that acts upon or with entities. An activity could be compiling code, running a test, or, crucially, an AI agent performing a task like "triage issue."
- **Agent**: An entity that bears some form of responsibility for an activity. An agent can be a person, a piece of software, or an organization.

While the standard prov:Agent is a powerful abstraction, the unique characteristics of agentic AI workflows demand a more specialized vocabulary. This need is addressed by the academic proposal known as **PROV-AGENT**, a formal extension to W3C PROV designed specifically for tracking AI agent interactions.[34] PROV-AGENT treats AI agent actions as first-class components within the provenance graph, allowing for a unified view that seamlessly integrates traditional software development activities with autonomous AI tasks.[34]

PROV-AGENT achieves this by introducing a set of specialized subclasses that capture the granular details of an agent's operation. These include AIAgent (a specific type of prov:Agent), AgentTool (representing the specific function or tool the agent invoked), AIModelInvocation (capturing the call to the underlying LLM), and entities for the Prompt and ResponseData.[34] This schema provides exactly the level of detail required to answer the critical questions of the provenance gap: which agent, using which tool, prompted which model with what data, to generate a specific output. Adopting PROV-AGENT as the core data model provides the semantic foundation upon which a verifiable system can be built.

Pillar 2: Process-Level Integrity with SLSA for AI

Having a standard data model for provenance is necessary, but it is not sufficient. The provenance data itself must be generated by a secure and trustworthy process. The second pillar addresses this by adapting a leading framework from the software supply chain security domain: Supply-chain Levels for Software Artifacts (SLSA), pronounced "salsa".[36]

SLSA is a security framework comprising a set of standards and controls to prevent tampering, improve integrity, and secure packages and infrastructure throughout the software supply chain.[36] It defines a series of assurance levels (from 0 to 3, with 4 planned) that provide increasing confidence in the integrity of a software artifact. The core idea is to generate authenticated, non-forgeable "provenance attestations" that describe exactly how an artifact was produced.[37]

The key to applying this framework to AI is to recognize that the AI/ML lifecycle is itself a

supply chain with analogous components. A crucial insight is that any transformation process within this lifecycle can be treated as a "build" in the SLSA model.[38] Data cleaning, feature engineering, model training, and fine-tuning are all "build" steps. The inputs to these steps, such as raw datasets or pre-trained base models, are the "dependencies".[38]

Extending this logic to agentic workflows, each autonomous action performed by an agent—such as "generate a code fix" or "review a pull request"—can be conceptualized as a verifiable build. Applying SLSA principles to this process means that the agent's execution environment (e.g., the GitHub Actions runner) must be hardened and isolated. More importantly, at the conclusion of its action, the agent must generate a signed SLSA provenance attestation. This attestation would verifiably document the "build" process: the inputs (the prompt, context files, tool configurations), the "builder" (the specific agent and model version), and the resulting "artifact" (the code commit or PR comment).[39] This provides process-level integrity, ensuring that the provenance record is a faithful representation of a secure and untampered execution.

Pillar 3: Cryptographic Identity and Attestation with DIDs and VCs

The third pillar provides the cryptographic foundation for identity and trust, enabling the creation of non-repudiable and verifiable records. This is achieved by leveraging two emerging W3C standards from the world of decentralized identity: Decentralized Identifiers (DIDs) and Verifiable Credentials (VCs).

- **Decentralized Identifiers (DIDs)**: A DID is a new type of globally unique identifier that is designed to be controlled by the entity it identifies, rather than being issued by a centralized registry.[41] DIDs are cryptographically verifiable; they are linked to a public key, allowing anyone to verify a signature created by the DID's controller.[42] By assigning a unique DID to each AI agent instance at the start of its lifecycle, we provide it with a persistent, non-repudiable identity that it can use to sign its actions and assertions.[43]
- **Verifiable Credentials (VCs)**: A VC is a standardized, tamper-evident digital credential containing a set of claims that an issuer makes about a subject.[45] The entire credential is digitally signed by the issuer, making it cryptographically verifiable.[47] In the context of agentic workflows, this model provides a powerful mechanism for attestation. For every significant action it performs, the AI agent (acting as its own issuer and subject, identified by its DID) can generate a VC. This VC would contain the detailed provenance information structured according to the PROV-AGENT schema. The agent then signs the VC with the private key associated with its DID.[47]

The result is a self-contained, cryptographically secure attestation for every atomic action. This transforms the provenance record from a simple log entry into a verifiable digital object.

Anyone can independently verify the authenticity of the VC by checking the agent's signature against its public DID, creating a decentralized and auditable trail of every action taken.[48] This combination of DIDs and VCs provides the final, crucial layer of cryptographic proof, moving the system from mere observation to genuine verifiability.

Comparative Analysis of Provenance and Supply Chain Security Frameworks for Agentic AI

To clarify how these distinct but complementary pillars form a cohesive solution, the following table provides a comparative analysis of their roles, capabilities, and limitations within the proposed framework.

Framework	Core Purpose	Granularity	Cryptographic Verifiability	Agent-Centricity	Role in Proposed Framework
OpenTelemetry	Observability & Debugging	Function-level traces, logs, metrics	None	Low	Foundational logging and signal collection
W3C PROV	Standardized Lineage Data Model	Entity-Activity-Agent relationships	Not inherent	Low (generic prov:Agent)	Core data schema for structuring provenance
PROV-AGENT	Agent-Specific Lineage Model	Prompt, Model, Tool, and Response level	Not inherent	High (designed for AI agents)	Schema extension to capture agentic details
SLSA for AI	Build Process Integrity	Build step verification	Signed Attestations	Moderate (adaptable to AI workflows)	Process integrity wrapper for secure execution

DIDs & VCs	Cryptographic Identity & Attestation	Individual, verifiable claims	Natively cryptographic	High (can model agents as subjects/issuers)	Identity, trust, and non-repudiation layer

This analysis demonstrates that no single technology is sufficient to solve the provenance crisis. OpenTelemetry provides the raw data but lacks structure and verifiability. W3C PROV and PROV-AGENT provide the necessary semantic structure but lack inherent security. SLSA secures the execution process but requires a robust identity mechanism. DIDs and VCs provide that cryptographic identity and the format for verifiable attestation. Only by integrating these five components—from signal collection to cryptographic proof—can a truly end-to-end, verifiable provenance framework be constructed.

A Multi-Layered Framework for Cryptographically Verifiable Provenance

By integrating the foundational pillars of standardized provenance models, process integrity frameworks, and cryptographic identity, it is possible to architect a cohesive, multi-layered solution that closes the provenance gap. This framework transforms provenance from a passive, after-the-fact metadata record into an active, verifiable asset that can be used to enforce policy and build trust. The architecture is composed of four distinct but interconnected layers, each building upon the last to create a comprehensive system for verifiable provenance.

Layer 1: Agent Identity and Lifecycle Management

The foundation of any accountability system is a robust and non-repudiable identity. The first layer of the framework establishes this for every autonomous agent.

Upon initialization within a secure execution environment, such as a GitHub Actions runner, every AI agent instance must be provisioned with a unique Decentralized Identifier (DID).[44] This DID, along with its associated cryptographic key pair, serves as the agent's immutable identity for the entire duration of its lifecycle. Unlike traditional identifiers that are assigned by a central authority, the agent itself controls its DID, ensuring self-sovereignty. The public key

associated with the DID is made available for others to verify signatures, while the private key is held securely within the execution environment and used to sign all subsequent attestations.

This cryptographic identity can be linked to the agent's operational context. For example, the DID document could contain metadata associating the agent with the specific GitHub repository, the workflow file that triggered it, the commit SHA, and the identity of the human user who initiated the run. The agent's capabilities and permissions can then be managed using standard Identity and Access Management (IAM) practices, where policies are attached to this unique, verifiable identity rather than a generic service account.[50] This ensures that every action can be traced back to a specific, uniquely identifiable agent instance.

Layer 2: Action Attestation with PROV-AGENT Verifiable Credentials

With a secure identity established, the second layer focuses on capturing and attesting to every significant action the agent performs. For each discrete, meaningful operation—such as analyzing an issue, calling an external tool, generating a code suggestion, or invoking an LLM—the agent must generate a W3C-compliant Verifiable Credential (VC).[47]

The structure, or *schema*, of this VC will be based on the PROV-AGENT model, providing a rich, standardized format for the claims it contains.[34] A typical action VC would include the following claims:

- issuer and credentialSubject: The agent's own DID, indicating that the agent is attesting to its own action.
- prov:agent: The DID of the agent performing the action.
- prov:activity: A unique identifier for the specific activity, such as triage_issue:run-123.
- prov:used: A list of identifiers for all input entities consumed by the activity. This would include the issue body, any context files like GEMINI.md, and, critically, a detailed Prompt object.
- AIModelInvocation: A structured object detailing the LLM call, including the AIModel name (e.g., gemini-2.5-pro), version, provider, and key parameters like temperature and top-k.
- prov:wasGeneratedBy: A list of identifiers for all output entities created, such as the ResponseData from the LLM or a newly generated code block.

Once this structured data is assembled, the entire VC is digitally signed by the agent using the private key associated with its DID. The result is a tamper-evident, non-repudiable attestation that serves as a verifiable receipt for the agent's action. This process fundamentally changes the nature of provenance. It is no longer a passive log entry whose integrity depends on the security of the logging system. Instead, it becomes a self-contained, cryptographically secure

digital asset. Its authenticity can be verified by anyone, at any time, simply by checking the signature against the agent's public DID. This makes provenance an active component of the trust infrastructure, enabling automated policy enforcement such as, "Do not merge this pull request unless its generating agent provides a VC attesting that it performed a security scan."

Layer 3: Artifact Binding and Supply Chain Integrity

The third layer ensures that the final artifacts produced by the agent are inextricably linked to their verifiable provenance. It is not enough to simply generate VCs; they must be securely bound to the outputs they describe.

This binding can be achieved through cryptographic hashing. When an agent produces an artifact—a code commit, a pull request comment, or a new issue—the hash of the corresponding provenance VC is embedded directly into the artifact's metadata. For example, a git commit could include a Provenance-VC-Hash trailer in its commit message. A comment posted to a GitHub issue could contain the hash within a hidden HTML comment tag. This creates an unbreakable link: the artifact points directly to its proof of origin.

To provide a holistic view of the entire workflow, the agentic process is wrapped within a SLSA-compliant build environment.[37] At the completion of the GitHub Actions job, a final SLSA provenance attestation is generated. This top-level attestation describes the overall execution environment and, crucially, lists the hashes of all the individual action VCs produced during the run as its "materials" or "dependencies." This creates a nested, verifiable structure: the SLSA attestation guarantees the integrity of the overall workflow run, while the embedded VCs provide granular, verifiable proof for each autonomous decision made within that run.

Layer 4: Verifiable Data Registry for Audit and Discovery

The final layer of the framework addresses the storage and accessibility of the generated provenance attestations. To be useful for auditing, compliance, and debugging, the signed VCs and SLSA attestations must be stored in a location that is both highly available and itself verifiable.

While a full permissioned blockchain could provide maximum immutability and decentralization, a more pragmatic and scalable solution for an ecosystem like GitHub would be a purpose-built, publicly auditable transparency log.[51] This could be implemented using technologies similar to Certificate Transparency, which is used to monitor and audit the

issuance of TLS certificates.

When an agent generates a VC, it would submit it to this Verifiable Data Registry. The registry would cryptographically verify the signature, add it to an append-only log (such as a Merkle tree), and return a signed receipt of inclusion. This provides proof that the VC was registered at a specific time and has not been altered since. This registry would serve as the central source of truth for all AI-driven activities on the platform, allowing developers, security auditors, and compliance officers to retrieve and verify the complete provenance of any AI-generated artifact simply by looking up its hash or the DID of the agent that created it.[53] This enables fully automated auditing, real-time compliance monitoring, and high-fidelity root cause analysis, completing the end-to-end framework for verifiable provenance.

Actionable Implementation Roadmap for Gemini CLI and GitHub Actions

Translating the multi-layered architectural framework into a practical reality requires a phased implementation plan. This roadmap outlines a series of concrete steps for platforms like Google's Gemini CLI and GitHub Actions, progressing from immediate, high-impact enhancements to a long-term vision of a fully verifiable AI supply chain. This approach allows for incremental value delivery while building towards the comprehensive solution.

Phase 1: Immediate Enhancements (Cognitive Scaffolds & Sequential Workflows)

The initial phase focuses on providing immediate, tangible improvements in transparency for human developers and laying the data groundwork for future automation. These steps directly address the user's initial suggestions for "Cognitive Scaffolds" and "Sequential Workflows."

- **Implement Human-Readable Audit Trails:** The first and most straightforward action is to mandate that all outputs from agentic workflows that are visible to users on GitHub (e.g., comments on pull requests, new issues, updates to issue labels) include a standardized, formatted markdown footer. This footer would serve as a human-readable audit trail, providing essential context at a glance. It should explicitly state:
    - The agent and version used (e.g., gemini-cli v0.1.18).
    - The specific model and version invoked (e.g., gemini-2.5-pro).
    - A list of primary context files accessed during the operation (e.g., Context

from:./.github/GEMINI.md,./src/main.py).
This simple change provides immediate value for debugging and manual review, offering a "cognitive scaffold" that helps developers understand the agent's context without needing to access complex backend logs.

- **Generate Machine-Readable Provenance Artifacts:** Concurrently, for every agentic action executed, the run-gemini-cli GitHub Action should be enhanced to generate a structured JSON file (e.g., provenance.json) as a standard workflow artifact. This file will not be visible in the UI by default but will be available for download and inspection. The schema of this JSON file should be an implementation of the **PROV-AGENT** model, capturing key details in a machine-readable format:
    - The full prompt sent to the model.
    - All model parameters used (temperature, top-p, etc.).
    - A list of all tools invoked and their parameters.
    - A comprehensive list of all files and repository paths accessed for context.
    - The raw response from the model.
    This step, enabled by a new configuration option in the workflow YAML, creates the foundational data layer. It makes detailed provenance information available for automated analysis and serves as the input for the attestation processes in subsequent phases.

Phase 2: Mid-Term Integration (Standardization and Attestation)

The second phase builds upon the data foundation of Phase 1 to introduce cryptographic verification and improve interoperability, moving from passive metadata to active attestations.

- **Drive Standardization of the Provenance Schema:** Google and GitHub, as industry leaders, should collaborate with standards bodies like the Open Source Security Foundation (OpenSSF) and the W3C to formalize the PROV-AGENT model (or a similar schema) as an industry standard for "AI Action Attestations".[54] This is a critical step to ensure a healthy, interoperable ecosystem where provenance data from different AI agent providers (Google, OpenAI, Anthropic, etc.) can be consumed and verified by a common set of tools.
- **Integrate Automated Attestation Generation:** The GitHub Actions platform should be enhanced to support automated, streamlined artifact attestation. This involves integrating tooling like Sigstore directly into the ecosystem. The workflow could be updated to automatically sign the provenance.json artifact generated in Phase 1. In this model, GitHub's own identity service would act as the Fulcio certificate authority, issuing a short-lived signing certificate that binds the workflow's identity to a public key. This elevates the provenance record from a simple file to a signed attestation, providing a guarantee that the record was produced by a specific, authenticated workflow run and

has not been tampered with. This would bring the system into compliance with SLSA Level 2 requirements.[36]

- **Enhance the GitHub User Interface:** To make this newly verifiable information accessible and useful, the GitHub UI must be updated to recognize and display it. An AI-generated comment, commit, or PR could feature a "Verified Agent Contribution" badge. Clicking this badge would open a modal window that parses the associated signed provenance.json attestation and presents the information in a clear, user-friendly format, detailing the model, prompt, and context used for that specific action. This makes the verifiable data accessible to all users, not just those willing to download and parse JSON artifacts.

Phase 3: Long-Term Vision (Cryptographically Verifiable AI Supply Chain)

The final phase realizes the full architectural vision, establishing a decentralized, cryptographically verifiable AI supply chain that provides the highest level of trust and accountability.

- **Implement Agent-Specific Decentralized Identifiers (DIDs):** The GitHub Actions platform would be evolved to support the generation of a unique, ephemeral DID for each individual agentic workflow run. Instead of signing attestations with the general identity of the workflow, the agent would use the private key associated with this run-specific DID. This provides a more granular and secure identity, isolating the cryptographic authority of each agentic process and enabling more precise policy enforcement and auditing.
- **Native Verifiable Credential (VC) Issuance:** The run-gemini-cli action (and similar tools from other vendors) would be upgraded to natively generate and sign W3C-compliant Verifiable Credentials for each atomic action. The action would no longer just output a JSON file; it would produce a set of cryptographically signed VCs, each conforming to the PROV-AGENT schema and signed by the run-specific DID. This achieves the core of the framework outlined in Section 5, creating self-contained, verifiable proofs of every decision.
- **Establish a Public Provenance Registry:** To complete the vision, GitHub could host a public, immutable transparency log where all provenance VCs generated by actions on the platform are published.[53] This registry, built on principles similar to Certificate Transparency, would provide a globally auditable record of AI activity. It would allow anyone to verify that a given VC was created at a certain time and has not been revoked or altered. This creates a fully auditable, cryptographically verifiable "AI supply chain" for every automated action taken on the platform, fulfilling the most advanced requirements for trust and transparency and aligning with academic proposals for end-to-end

verifiable AI pipelines.[30]

Future Horizons and Unresolved Challenges

The implementation of a framework for verifiable provenance is not the end of the journey but rather the beginning. Establishing this foundational layer of trust opens up new possibilities for more advanced, intelligent, and responsible AI systems. However, it also brings to light a new set of complex challenges that will define the next frontier of research and development in this domain.

The Intersection of Provenance and Explainable AI (XAI)

One of the most promising future directions is the deep integration of provenance with the field of Explainable AI (XAI). While often conflated, provenance and explainability are distinct but synergistic concepts. Provenance provides a factual, verifiable record of *what happened* in a system's execution. Explainability aims to provide a human-understandable rationale for *why it happened*. The rich, structured, and verifiable provenance data generated by the proposed framework is the essential raw material for producing high-quality explanations.

Current XAI techniques often focus on "feature-based" explanations, which identify the input features that were most influential in a model's decision. Provenance data enables a complementary and arguably more powerful approach known as "trace-based explanation".[56] A trace-based explanation uses the provenance graph to construct a narrative of the entire decision-making process, from the initial data inputs through each intermediate step of reasoning and tool use, to the final output.

Future research will likely focus on developing interactive XAI systems that act as "explanation agents".[56] These agents could consume the provenance graph of a decision and engage in a natural language dialogue with a user. They could tailor the explanation to the user's level of technical expertise, allowing a project manager to receive a high-level summary while an engineer could drill down into specific model parameters or API call results.[58] This fusion of verifiable provenance and interactive explanation is key to building systems that are not just auditable but truly understandable.

Open Challenges and Research Directions

Despite the promise of this vision, several significant challenges must be addressed through further research and engineering.

- **Performance Overhead and Scalability:** The process of generating, signing, and registering a cryptographic credential for every single agent action could introduce significant performance overhead. The latency from cryptographic operations and network calls to a registry could slow down workflows, negating some of the productivity benefits of AI automation. Future work must focus on developing lightweight cryptographic schemes, efficient batching mechanisms for attestations, and highly scalable registry architectures to ensure that verifiability does not come at an unacceptable performance cost.
- **Provenance of Multi-Agent Conversations:** The proposed framework is well-suited for tracking the discrete, atomic actions of individual agents. A far more complex challenge is capturing the provenance of an emergent idea or a collaborative decision that arises from a long, dynamic conversation between multiple, potentially heterogeneous agents.[28] This requires moving beyond a simple activity log to a model that can represent the flow of influence, the establishment of trust, and the synthesis of information within a multi-agent system.[60] This is a frontier research problem at the intersection of distributed systems, game theory, and AI.
- **Standardization and Ecosystem Adoption:** The success of a verifiable AI supply chain is contingent upon broad industry adoption of common standards. Technologies like DIDs, VCs, and a standardized provenance schema like PROV-AGENT must be embraced by all major players—cloud providers, AI model developers, and platform operators.[62] Overcoming the natural tendency towards vendor-specific, proprietary solutions and fostering a collaborative, open ecosystem will be a significant socio-technical challenge that requires concerted effort from standards bodies, industry consortia, and open-source communities.
- **Privacy and Confidentiality:** A detailed, immutable provenance trail, while essential for accountability, also creates significant privacy risks. These records could contain sensitive information, including proprietary source code, internal business logic, or personally identifiable information (PII) from issue reports.[9] Publishing such information to a public registry is often not feasible. Therefore, a critical area of research is the development of privacy-preserving provenance techniques. Technologies like zero-knowledge proofs could allow an agent to prove that its action complied with certain policies (e.g., "no PII was used as input," "a security scan was performed") without revealing the confidential details of the action itself.[30] Balancing the need for transparency with the imperative of confidentiality will be a central challenge in deploying these systems at scale.

Conclusion: Building the Foundations for Trustworthy AI

The integration of autonomous, agentic AI into the core of the software development lifecycle is an inevitable and transformative evolution. These technologies promise to amplify developer productivity, accelerate innovation, and manage the ever-increasing complexity of modern software. However, this promise is shadowed by the profound risk of opaque, untrustworthy systems that undermine security, violate regulations, and erode the very foundations of accountability.

To proceed safely and responsibly into this new era, we must move beyond a paradigm of blind trust and build a new infrastructure for verifiable proof. Proactively establishing a robust framework for cryptographic provenance is not an optional feature or a compliance afterthought; it is a foundational requirement for the future of software engineering. The multi-layered architecture and phased roadmap presented in this report provide a clear, actionable path forward. By combining standardized data models, process integrity controls, and cryptographic identity, platforms like GitHub and Google can lead the industry in constructing this critical new layer of the development stack.

The ultimate goal is to create an ecosystem where every contribution, whether human or AI-driven, is accompanied by an immutable, verifiable record of its origin and lineage. This is the bedrock upon which trustworthy AI can be built, ensuring that the future of software development is not only more efficient and powerful but also more secure, compliant, and accountable. The work to build this future must begin now, embedding trust and governance into the very first line of code written by our new autonomous teammates.[64]

Works cited

1. GitHub Copilot · Your AI pair programmer, accessed on September 2, 2025, https://github.com/features/copilot
2. Agentic Workflows - GitHub Next, accessed on September 2, 2025, https://githubnext.com/projects/agentic-workflows/
3. Meet your new AI coding teammate: Gemini CLI GitHub Actions - The Keyword, accessed on September 2, 2025, https://blog.google/technology/developers/introducing-gemini-cli-github-actions/
4. How Gemini CLI GitHub Actions is Changing Developer Workflows - DevOps.com, accessed on September 2, 2025, https://devops.com/how-gemini-cli-github-actions-is-changing-developer-workflows/
5. How to Use Gemini CLI GitHub Actions for Free - Apidog, accessed on

September 2, 2025, https://apidog.com/blog/gemini-cli-github-actions/
6. PROV-DM: The PROV Data Model - W3C, accessed on September 2, 2025, https://www.w3.org/TR/prov-dm/
7. Recording provenance of workflow runs with RO-Crate | PLOS One - Research journals, accessed on September 2, 2025, https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0309210
8. What is Data Provenance? | IBM, accessed on September 2, 2025, https://www.ibm.com/think/topics/data-provenance
9. Data Provenance: A Beginner's Guide - TechnologyAdvice, accessed on September 2, 2025, https://technologyadvice.com/blog/business-intelligence/data-provenance/
10. Can AI really code? Study maps the roadblocks to autonomous software engineering, accessed on September 2, 2025, https://news.mit.edu/2025/can-ai-really-code-study-maps-roadblocks-to-autonomous-software-engineering-0716
11. AI & the GDPR: Regulating the minds of machines | DigiLinks | Linklaters, accessed on September 2, 2025, https://www.linklaters.com/insights/blogs/digilinks/ai-and-the-gdpr-regulating-the-minds-of-machines
12. GitHub Agentic Workflows - dsyme on software, accessed on September 2, 2025, https://dsyme.net/2025/08/27/github-agentic-workflows/
13. githubnext/gh-aw: GitHub Agentic Workflows from GitHub Next, accessed on September 2, 2025, https://github.com/githubnext/gh-aw
14. Goodbye API Keys: Gemini CLI GitHub Actions with Workload Identity Federation - Medium, accessed on September 2, 2025, https://medium.com/google-cloud/goodbye-api-keys-gemini-cli-github-actions-with-workload-identity-federation-6d4fae9e936b
15. Provenance in Scientific Workflow Systems - IEEE Computer Society, accessed on September 2, 2025, http://sites.computer.org/debull/a07dec/susan.pdf
16. (PDF) Provenance in Scientific Workflow Systems - ResearchGate, accessed on September 2, 2025, https://www.researchgate.net/publication/220283434_Provenance_in_Scientific_Workflow_Systems
17. Provenance Collection Support in the Kepler Scientific Workflow System - San Diego Supercomputer Center, accessed on September 2, 2025, https://users.sdsc.edu/~altintas/KeplerProvenanceRecorder.pdf
18. Right to explanation - Wikipedia, accessed on September 2, 2025, https://en.wikipedia.org/wiki/Right_to_explanation
19. AI and the right to an explanation - Blog - DPO Centre, accessed on September 2, 2025, https://www.dpocentre.com/ai-and-the-right-to-an-explanation/
20. AI Act - data governance and compliance strategy implications in Pharma, accessed on September 2, 2025, https://www.europeanpharmaceuticalreview.com/article/264445/ai-act-data-governance-and-compliance-strategy-implications-in-pharma/
21. GDPR and Large Language Models: Technical and Legal Obstacles - MDPI,

accessed on September 2, 2025, https://www.mdpi.com/1999-5903/17/4/151
22. THE RIGHT TO EXPLANATION IN THE PROCESSING OF PERSONAL DATA WITH THE USE OF AI SYSTEMS, accessed on September 2, 2025, https://ijlcw.emnuvens.com.br/revista/article/view/53
23. 5 AI Auditing Frameworks to Encourage Accountability - AuditBoard, accessed on September 2, 2025, https://auditboard.com/blog/ai-auditing-frameworks
24. AI and intellectual property rights - Dentons, accessed on September 2, 2025, https://www.dentons.com/ru/insights/articles/2025/january/28/ai-and-intellectual-property-rights
25. The interaction between intellectual property laws and AI: Opportunities and challenges, accessed on September 2, 2025, https://www.nortonrosefulbright.com/en/knowledge/publications/c6d47e6f/the-interaction-between-intellectual-property-laws-and-ai-opportunities-and-challenges
26. Generative AI: Navigating Intellectual Property - WIPO, accessed on September 2, 2025, https://www.wipo.int/documents/d/frontier-technologies/docs-en-pdf-generative-ai-factsheet.pdf
27. Will copyright law enable or inhibit generative AI? - The World Economic Forum, accessed on September 2, 2025, https://www.weforum.org/stories/2024/01/cracking-the-code-generative-ai-and-intellectual-property/
28. Agent Tracing for Debugging Multi-Agent AI Systems - Maxim AI, accessed on September 2, 2025, https://www.getmaxim.ai/articles/agent-tracing-for-debugging-multi-agent-ai-systems/
29. To all of you making agents. How are you handling agents or multi-agent systems that get really complicated? - Reddit, accessed on September 2, 2025, https://www.reddit.com/r/AI_Agents/comments/1m6avli/to_all_of_you_making_agents_how_are_you_handling/
30. Verifiable Chains of Custody: Securing the AI Supply Chain with ..., accessed on September 2, 2025, https://uplatz.com/blog/verifiable-chains-of-custody-securing-the-ai-supply-chain-with-watermarking-and-cryptographic-provenance/
31. Why is AI Provenance Taken So Lightly? : r/ArtificialInteligence - Reddit, accessed on September 2, 2025, https://www.reddit.com/r/ArtificialInteligence/comments/1liggjm/why_is_ai_provenance_taken_so_lightly/
32. What is AI transparency? A comprehensive guide - Zendesk, accessed on September 2, 2025, https://www.zendesk.com/blog/ai-transparency/
33. The PROV XML Schema - W3C, accessed on September 2, 2025, https://www.w3.org/TR/prov-xml/
34. PROV-AGENT: Unified Provenance for Tracking AI Agent ... - arXiv, accessed on September 2, 2025, https://arxiv.org/abs/2508.02866
35. PROV-AGENT: Unified Provenance for Tracking AI Agent Interactions in Agentic

Workflows This manuscript has been authored by UT-Battelle, LLC, under contract DE-AC05-00OR22725 with the US Department of Energy (DOE). The publisher, by accepting the article for publication, acknowledges that the U.S. Government retains a non-exclusive - arXiv, accessed on September 2, 2025, https://arxiv.org/html/2508.02866v2
36. Understanding the SLSA framework - Buildkite, accessed on September 2, 2025, https://buildkite.com/resources/blog/understanding-the-slsa-framework/
37. Enhance build security and reach SLSA Level 3 with GitHub Artifact Attestations, accessed on September 2, 2025, https://github.blog/enterprise-software/devsecops/enhance-build-security-and-reach-slsa-level-3-with-github-artifact-attestations/
38. Document how to do SLSA for ML and highlight gaps #978 - GitHub, accessed on September 2, 2025, https://github.com/slsa-framework/slsa/issues/978
39. Increasing transparency in AI security, accessed on September 2, 2025, https://security.googleblog.com/2023/10/increasing-transparency-in-ai-security.html
40. Atlas: A Framework for ML Lifecycle Provenance & Transparency - arXiv, accessed on September 2, 2025, https://arxiv.org/html/2502.19567v1
41. Why AI Agents Need Verified Digital Identities - Identity.com, accessed on September 2, 2025, https://www.identity.com/why-ai-agents-need-verified-digital-identities/
42. www.identity.com, accessed on September 2, 2025, https://www.identity.com/why-ai-agents-need-verified-digital-identities/#:~:text=A%20decentralized%20identifier%20is%20a,and%20where%20it%20came%20from.
43. The AI Agent Needs a Wallet—Why Decentralized Identity Matters More Than Ever, accessed on September 2, 2025, https://www.arcblock.io/content/blog/en/ai-decentralized-identity
44. Securing AI Agent Communication: Decentralized Identity & Protocol - Synergetics.ai, accessed on September 2, 2025, https://synergetics.ai/securing-ai-agent-communication-decentralized-identity-protocol/
45. Verifiable Credentials: The Foundation of an Ethical AI-Powered Future - Gobekli.io, accessed on September 2, 2025, https://gobekli.io/verifiable-credentials-the-foundation-of-an-ethical-ai-powered-future/
46. Verifiable Credentials: The Ultimate Guide 2025 - Dock Labs, accessed on September 2, 2025, https://www.dock.io/post/verifiable-credentials
47. Verifiable Credentials: A Deep Dive for the Agentic AI Era - Shankar's Blog, accessed on September 2, 2025, https://shankarkumarasamy.blog/2025/02/28/verifiable-credentials-a-deep-dive-for-the-agentic-ai-era/
48. What Is Verifiable AI? A Guide to Transparency and Trust in AI - Identity.com, accessed on September 2, 2025, https://www.identity.com/what-is-verifiable-ai-a-guide-to-transparency-and-trus

t-in-ai/
49. Origin of Verifiable AI:The Interplay of AI and Verifiable Credentials | cheqd, accessed on September 2, 2025, https://cheqd.io/blog/origin-of-verifiable-aithe-interplay-of-ai-and-verifiable-credentials/
50. IAM Best Practices for AI Agents - Ping Identity, accessed on September 2, 2025, https://www.pingidentity.com/en/resources/identity-fundamentals/agentic-ai/iam-best-practices-ai-agents.html
51. (PDF) Enhancing supply chain security and transparency with ai and ..., accessed on September 2, 2025, https://www.researchgate.net/publication/385382441_Enhancing_supply_chain_security_and_transparency_with_ai_and_blockchain_integration_for_future-proof_solutions
52. (PDF) Building Trust with Blockchain AI-Enhanced Verification in Supply Chains, accessed on September 2, 2025, https://www.researchgate.net/publication/388029788_Building_Trust_with_Blockchain_AI-Enhanced_Verification_in_Supply_Chains
53. NIST rolls out meta-framework, reference model to boost traceability and cybersecurity across manufacturing supply chains - Industrial Cyber, accessed on September 2, 2025, https://industrialcyber.co/nist/nist-rolls-out-meta-framework-reference-model-to-boost-traceability-and-cybersecurity-across-manufacturing-supply-chains/
54. A Practical Guide for Building Robust AI/ML Pipeline Security, accessed on September 2, 2025, https://openssf.org/wp-content/uploads/2025/08/OpenSSF_MLSecOps_Whitepaper.pdf
55. A Framework for Cryptographic Verifiability of End-to-End AI Pipelines - arXiv, accessed on September 2, 2025, https://arxiv.org/html/2503.22573v1
56. Co-constructing Explanations for AI Systems using Provenance - arXiv, accessed on September 2, 2025, https://arxiv.org/html/2507.17761v1
57. What information is required for explainable AI? a provenance-based research agenda and future challenges, accessed on September 2, 2025, https://research.monash.edu/en/publications/what-information-is-required-for-explainable-ai-a-provenance-base
58. Provenance Documentation to Enable Explainable and Trustworthy AI - UIdaho, accessed on September 2, 2025, https://objects.lib.uidaho.edu/etd/pdf/Kale_idaho_0089E_12560.pdf
59. Provenance documentation to enable explainable and trustworthy AI: A literature review - NSF Public Access Repository, accessed on September 2, 2025, https://par.nsf.gov/servlets/purl/10328231
60. Trust Decision-Making in Multi-Agent Systems - IJCAI, accessed on September 2, 2025, https://www.ijcai.org/Proceedings/11/Papers/031.pdf
61. (PDF) Data provenance in multi-agent systems: relevance, benefits and research opportunities - ResearchGate, accessed on September 2, 2025, https://www.researchgate.net/publication/329328505_Data_provenance_in_multi

-agent_systems_relevance_benefits_and_research_opportunities
62. AI Standards Transparency: Decoding the Next Corporate Disclosures Frontier, accessed on September 2, 2025, https://www.cigionline.org/publications/ai-standards-transparency-decoding-the-next-corporate-disclosures-frontier/
63. AI Risk Management Framework | NIST, accessed on September 2, 2025, https://www.nist.gov/itl/ai-risk-management-framework
64. The Future of Trustworthy AI - CRA - Computing Research Association, accessed on September 2, 2025, https://cra.org/events/the-future-of-trustworthy-ai/
65. The Future of AI - Why Trust and Governance Matter - SAS, accessed on September 2, 2025, https://www.sas.com/en_us/webinars/trust-and-governance-matter.html